

CPE 241 Database Systems (Module 2)

Mock Exam (ข้อสอบจำลอง)

เวลาสอบ: 3 ชั่วโมง

คะแนนเต็ม 60 คะแนน

Part 1: Multiple Choice Questions (20 Marks)

คำชี้แจง: เลือกคำตอบที่ถูกต้องที่สุด (ข้อละ 1 คะแนน)

ข้อ 1-8 อิงตามหัวข้อที่ออกสอบจริงในปีที่แล้ว ส่วนข้อ 9-20 เป็นเนื้อหาวิเคราะห์เพิ่มเติมตามสไลด์

- ข้อใดกล่าวถึงคุณสมบัติของ Normal Form ในแต่ละระดับได้ถูกต้องที่สุด?
 - 1NF: ข้อมูลทุกช่องต้องเป็น Atomic value และอนุญาตให้มี Multivalued attribute ได้
 - 2NF: ต้องไม่มี Partial Dependency เกิดขึ้นกับตารางที่มี Composite Primary Key
 - 3NF: ทุกตัวกำหนดค่า (Determinant) จะต้องมีความสัมพันธ์เป็น Candidate Key เสมอ
 - BCNF: อนุญาตให้มี Transitive Dependency ได้ หากตัวกำหนดค่าเป็น Primary Key
- ภาษาสอบถาม (Query Language) ในข้อใด ที่จัดเป็นภาษาแบบ Procedural (ต้องระบุขั้นตอนการทำงาน)?
 - SQL
 - TRC (Tuple Relational Calculus)
 - DRC (Domain Relational Calculus)
 - RA (Relational Algebra)
- จากสมการ Tuple Relational Calculus (TRC) ด้านล่าง มีความหมายตรงกับข้อใด?

$$\{t.\text{Fname} \mid t \in \text{Employee} \wedge \forall s \in \text{Employee}(t.\text{Salary} \geq s.\text{Salary})\}$$

- หาชื่อพนักงานที่มีเงินเดือนน้อยที่สุด
 - หาชื่อพนักงานที่มีเงินเดือน “สูงที่สุด” (Highest-paid employee)
 - หาชื่อพนักงานทุกคนที่มีเงินเดือนเท่ากัน
 - หาชื่อพนักงานที่มีเงินเดือนมากกว่าพนักงานทุกคนในแผนก 5
- การดำเนินการ Join ข้อมูลในรูปแบบใด ที่ทำให้เกิดค่าว่าง (Null value) ในตารางผลลัพธ์ได้เสมอหากข้อมูลไม่ตรงกัน?
 - Natural Join
 - Theta Join
 - Outer Join (เช่น Left, Right, Full)
 - Equi-Join

5. จากสมการ Relational Algebra ด้านล่าง จัดเป็นเทคนิค Query Optimization แบบใด?

$$\pi_{s.name, g.grade}(\sigma_{g.grade='A'}(students \bowtie grades)) \Rightarrow \pi_{s.name, g.grade}(students \bowtie \sigma_{g.grade='A'}(grades))$$

- Projection Pushdown
 - Predicate Pushdown
 - Subquery Flattening
 - Left-deep Tree Transformation
6. การแบ่งแยกตารางแนวนอนโดยตัดเฉพาะบางแถว (Rows) ที่สอดคล้องกับเงื่อนไข เช่น แยกเฉพาะนักศึกษาปี 1 ออกมาอีกตารางหนึ่ง เรียกว่าอะไร?
- Vertical Partitioning
 - Horizontal Partitioning
 - Range Partitioning
 - Hash Partitioning
7. ผลกระทบหลักจาก การสร้าง Index บนคอลัมน์ในระบบฐานข้อมูลคือข้อใด?
- ทำให้อ่านข้อมูลเร็วขึ้น (SELECT) แต่อาจทำให้การเขียนช้าลง (INSERT / UPDATE)
 - ฐานข้อมูลเปลี่ยนเป็นระดับ 3NF ทันที
 - ค่าของตัวแปรจะเป็นเอกลักษณ์เสมอ
 - ประหยัดพื้นที่ใน Hard Disk
8. กำหนดให้ Table A มีข้อมูลอยู่ทั้งหมด 100 Records การดำเนินการทางคณิตศาสตร์ (Relational Algebra) ในข้อใด ที่จะทำได้จำนวน Records ผลลัพธ์ “เยอะที่สุด” ?
- $A \cup A$ (Union)
 - $A \cap A$ (Intersection)
 - $A - A$ (Set Difference)
 - $A \times A$ (Cartesian Product)
9. หากต้องการกรองข้อมูลโดยเช็คค่า “มีผลลัพธ์ใดๆ ถูกส่งกลับมาจาก Subquery หรือไม่” ควรใช้คีย์เวิร์ดใดใน SQL?
- IN
 - EXISTS
 - > ALL
 - ANY

10. คำสั่ง SELECT * ส่งผลเสียต่อการทำงานของระบบอย่างไรเมื่อเทียบกับระบบชื่อคอลัมน์?
- ทำให้เกิด Error บ่อย
 - เพิ่ม Data Transfers เกินความจำเป็น และลดประสิทธิภาพของ Projection Pushdown
 - ทำให้ไม่สามารถใช้ GROUP BY ได้
 - ทำให้เกิด Full Table Scan เสมอ
11. ปัญหา Modification Anomaly ประเภท “Deletion Anomaly” เกิดขึ้นในกรณีใด?
- ลบพนักงานคนสุดท้ายของแผนกทิ้ง ทำให้ข้อมูลของ “แผนก” นั้นสูญหายไปด้วย
 - แก้ไขข้อมูลผู้จัดการแผนก แต่ต้องตามไปแก้ไขข้อมูลของพนักงานทุกคนในแผนกนั้น
 - ไม่สามารถเพิ่มพนักงานใหม่ได้เพราะยังไม่ได้ระบุแผนก
 - ถูกทุกข้อ
12. ความแตกต่างระหว่าง WHERE และ HAVING คืออะไร?
- WHERE ทำงานหลัง GROUP BY, HAVING ทำงานก่อน GROUP BY
 - WHERE กรองข้อมูลก่อนการจัดกลุ่ม (Group By) ซึ่งช่วยเพิ่ม Performance, HAVING กรองหลังจัดกลุ่ม
 - HAVING ไม่สามารถใช้ฟังก์ชัน Aggregate ได้
 - ทั้งสองคำสั่งสามารถใช้สลับกันได้ 100%
13. หากนำตรรกศาสตร์ Three-valued logic (3VL) ไปประมวลผลนิพจน์ NOT (UNKNOWN OR FALSE) AND TRUE ผลลัพธ์จะได้ค่าใด?
- TRUE
 - FALSE
 - UNKNOWN
 - NULL
14. ทำไมใน SQL จึงห้ามใช้เครื่องหมาย = ในการเปรียบเทียบค่า NULL (เช่น WHERE grade = NULL)?
- เพราะ NULL เป็นตัวแปรประเภท String
 - เพราะ NULL หมายถึง “ไม่รู้ค่า (Unknown)” การเปรียบเทียบจึงไม่ได้ค่าความเป็นจริง ต้องใช้ IS NULL แทน
 - เพราะจะทำให้เกิด Syntax Error ทันที
 - สามารถใช้ได้ ไม่มีข้อห้าม
15. BCNF (Boyce-Codd Normal Form) ถูกสร้างขึ้นมาเพื่ออุดช่องโหว่ของ 3NF ในกรณีใด?
- กรณีที่มี Multivalued attribute
 - กรณีที่ตารางมี Primary Key เพียงคอลัมน์เดียว
 - กรณีที่มีตัวกำหนดค่า (Determinant) ซึ่งฝั่งซ้ายของลูกศรไม่ได้เป็น Candidate Key
 - กรณีที่ตารางมีจำนวนแถวมากเกินไป

16. เมื่อเขียน Nested Query ด้วยคำสั่ง NOT IN หากผลลัพธ์ของ Subquery มีค่า NULL ปะปนอยู่ ผลลัพธ์ของการกรรานั้นจะเป็นอย่างไร?
- ประเมินผลเป็นจริง (TRUE) สำหรับค่าที่ระบุ
 - ประเมินผลเป็นเท็จ (FALSE) หรือ UNKNOWN ทำให้ไม่ระบุข้อมูลใดๆ กลับมาเลย
 - ระบบจะข้ามค่า NULL ไปและเปรียบเทียบเฉพาะค่าที่มีอยู่จริง
 - เกิดข้อผิดพลาด (Syntax Error) ทันที
17. การใช้คำสั่ง CREATE VIEW มีประโยชน์อย่างไรต่อ Database Administrator?
- ช่วยเร่งความเร็วในการ INSERT ข้อมูล
 - ช่วยซ่อนคอลัมน์หรือข้อมูลที่ละเอียดอ่อนจากผู้ทั่วไป (Security)
 - ทำให้เกิด Physical Table ใหม่ในฮาร์ดดิสก์
 - ช่วยลดขนาดของ Database ลงครึ่งหนึ่ง
18. คำสั่ง SQL ข้อใดที่สามารถใช้เชื่อมผลลัพธ์จาก 2 ควิรีเข้าด้วยกัน โดย “ตัดข้อมูลที่ซ้ำกันทิ้ง” อัตโนมัติ?
- UNION ALL
 - UNION
 - CROSS JOIN
 - EXCEPT
19. หากเราต้องการให้ควิรีหา “พนักงานที่เงินเดือนมากกว่าทุกคนในแผนก 5” โครงสร้าง Subquery ต้องใช้เครื่องหมายใด?
- > ANY
 - IN
 - > ALL
 - = SOME
20. ตัวแปรใน Domain Relational Calculus (DRC) อ้างอิงถึงสิ่งใด?
- ทั้งแถวข้อมูล (Tuple)
 - ค่าคงที่ทางคณิตศาสตร์
 - ค่าของคอลัมน์ (Column / Domain values)
 - ชื่อของตาราง

Part 2: Short Answer Questions (20 Marks)

คำชี้แจง: ตอบคำถามด้วยคำสั้นๆ หรือวลีแบบกระชับ (ข้อละ 2 คะแนน)

1. ความแตกต่างหลักระหว่างการประมวลผลคำสั่ง IN และ EXISTS เมื่อใช้ใน Nested Query คืออะไร?
2. สมมติว่าต้องการหาพนักงานที่มีเงินเดือน “มากกว่าพนักงานทุกคน” ในแผนกที่ 5 ควรใช้เครื่องหมายการเปรียบเทียบใดควบคู่กับ Sub-query?
3. ในคำสั่ง SQL การใช้ = ANY มีความหมายและหลักการทำงานเทียบเท่ากับการใช้คีย์เวิร์ดใดใน Nested Query?
4. การทำ Correlated Subquery มีลักษณะการทำงานที่ทำให้ซ้ำกว่า Subquery ปกติอย่างไร? อธิบายสั้นๆ
5. หากต้องการหาข้อมูลของแผนกที่ “ไม่มีพนักงานอยู่เลย” โดยใช้ Nested Query ควรใช้รูปแบบคำสั่งใดถึงจะเหมาะสมที่สุด?
6. ข้อควรระวังเมื่อใช้ CREATE VIEW แล้วต้องการออกแบบให้เป็น Updatable View (สามารถปรับปรุงข้อมูลกลับไปยังตารางหลักได้) คือข้อจำกัดใดบ้าง?
7. ในกระบวนการ Normalization ปัญหา “Repeating Groups” หรือช่องข้อมูลที่เก็บค่าหลายค่า (Multivalued) จะขัดกับกฎของ Normal Form ระดับใด?
8. จงอธิบายสั้นๆ ว่า Transitive Dependency ที่คอยขัดกับกฎของ 3NF มีลักษณะการพึ่งพาอย่างไร?
9. BCNF ถือเป็นรูปแบบที่เข้มงวดกว่า 3NF ตารางที่สมบูรณ์ในระดับ 3NF แล้วจะมีโอกาสพังและ “ไม่ผ่าน” ระดับ BCNF ในกรณีใด?

10. การสร้าง Index จำนวนมากบนตารางเดียวกัน มีผลเสียต่อระบบฐานข้อมูลในด้านใดบ้าง? (ระบุอย่างน้อย 2 ข้อ)

Part 3: Long Questions (20 Marks)

คำชี้แจง: ข้อแรกเป็น Complex Query 4 แบบ, ข้อสองเป็น Normalization 1NF -> 3NF

ข้อที่ 1: Complex Query Part (10 คะแนน)

สถานการณ์: กำหนดโครงสร้างฐานข้อมูล COMPANY ของบริษัทแห่งหนึ่ง ดังนี้ (อิงจาก COMPANY Database Schema)

EMPLOYEE (<u>Ssn</u> , Fname, Minit, Lname, Bdate, Address, Sex, Salary, Super_ssn, Dno)
DEPARTMENT (<u>Dnumber</u> , Dname, Mgr_ssn, Mgr_start_date)
DEPT_LOCATIONS (<u>Dnumber</u> , <u>Dlocation</u>)
PROJECT (<u>Pnumber</u> , Pname, Plocation, Dnum)
WORKS_ON (<u>Essn</u> , <u>Pno</u> , Hours)
DEPENDENT (<u>Essn</u> , <u>Dependent_name</u> , Sex, Bdate, Relationship)

คำชี้แจง: ให้เขียนคำตอบเป็นสมการ/คำสั่ง RA, TRC, DRC และ SQL สำหรับแต่ละข้อย่อยต่อไปนี้ โดย (โจทย์แต่ละข้อย่อยและพื้นที่ตอบให้อยู่ในหน้าเดียวกัน)

1.1 (3 คะแนน) หาชื่อ (Fname) และนามสกุล (Lname) ของพนักงานที่ทำงานในโปรเจกต์ชื่อ 'ProductX'

RA:

TRC:

DRC:

SQL:

1.2 (4 คะแนน) หาชื่อ (Fname) และนามสกุล (Lname) ของพนักงาน ที่ทำงานใน ทุกๆ โปรเจกต์ (All projects) ที่ควบคุมโดยแผนก 5 (Dnum = 5)

RA: (คำใบ้: ประยุกต์ใช้ตรรกะ Division)

TRC:

DRC:

SQL: (คำใบ้: ใช้ *NOT EXISTS* ซ้อน *NOT EXISTS*)

1.3 (3 คะแนน) หาชื่อ (Fname) และนามสกุล (Lname) ของพนักงานที่ ไม่มีผู้อยู่ในอุปการะ (No Dependents)

RA:

TRC:

DRC:

SQL:

ข้อที่ 2: Normalization (10 คะแนน)

สถานการณ์: บริษัทผลิตวิดีโอเกมในต่างประเทศได้เก็บข้อมูลของ ผู้พัฒนา-ตัวเกม-ควสย่อย (Studio-Game-Quest) ลงใน Report Table เดียวแบบไม่ได้จัดรูปแบบ (Unnormalized Data) ส่งผลให้เกิดปัญหาที่ระบบฐานข้อมูล ข้อมูลตัวอย่างที่ถูกดึงออกมาแสดงดังตาราง GAME_QUEST_STUDIO:

Studio	Country	Specialty	GameTitle	RelYear	Publisher	GameSize	QuestTitle	QuestNo	QuestType	RewardEXP
Bethesda	USA	RPG	Skyrim	2011	Zenimax	12GB	Unbound	1	Main	100
Bethesda	USA	RPG	Skyrim	2011	Zenimax	12GB	Before the Storm	2	Main	200
Bethesda	USA	RPG	Skyrim	2011	Zenimax	12GB	Golden Claw	7	Side	150
CDPR	Poland	ARPG	Witcher 3	2015	Bandai	35GB	Lilac and Gooseberries	2	Main	250
CDPR	Poland	ARPG	Witcher 3	2015	Bandai	35GB	Beast of White Orchard	6	Side	300
Larian	Belgium	CRPG	Baldur's Gate 3	2023	Larian	120GB	Escape the Nautiloid	1	Main	400
Larian	Belgium	CRPG	Baldur's Gate 3	2023	Larian	120GB	Explore ruins	12	Side	500

หมายเหตุ: Primary Key หลักของตารางนี้คือ Composite Key {GameTitle, QuestNo}

Functional Dependencies (FDs) ที่ถูกระบุจาก Business Rules:

- FD1: {GameTitle, QuestNo} → QuestTitle, QuestType, RewardEXP (รหัสเคลสของเกมนั้นๆ ระบุนามและรายละเอียดเคลสได้)
- FD2: GameTitle → Studio, RelYear, Publisher, GameSize (ชื่อเกม ระบุนามผู้สร้าง, ปีที่วางจำหน่าย, ผู้จัดจำหน่าย และขนาดเกมได้)
- FD3: Studio → Country, Specialty (ชื่อสตูดิโอ ระบุนามที่ตั้งประเทศ และแนวเกมที่ได้เล่น)

คำถาม:

2.1 (3 คะแนน) ตรวจสอบสถานะ Normal Form:

ตาราง GAME_QUEST_STUDIO ปัจจุบันจัดว่าอยู่ใน Normal Form ระดับใด? ชัดกับกฎข้อใดในระดับถัดไป? (1NF, 2NF, หรือ 3NF)

ให้อธิบายเหตุผลโดยอ้างอิงถึงประเภท Dependency จาก FDs ด้านบน

2.2 (3 คะแนน) ทำให้เป็น 2NF:

จงแสดงการแยกตารางให้บรรลุ Second Normal Form (2NF) ตารางใดบ้างที่ถูกแยกออกมา? (ให้เขียนชื่อตาราง, ชัดเส้นใต้ Primary Key และระบุ Foreign Key ให้ชัดเจน)

2.3 (4 คะแนน) ทำให้เป็น 3NF:

จากผลลัพธ์ในข้อ 2.2 จงดำเนินการแยกตารางต่อจนบรรลุ **Third Normal Form (3NF)** อย่างสมบูรณ์ ให้เขียน Schema ของตารางทั้งหมดที่ได้เป็นผลลัพธ์สุดท้าย พร้อมระบุ Data ตัวอย่างในตารางที่ถูกสร้างใหม่ (เฉพาะตารางที่เกิดจากการแก้ Transitive Dependency)

เฉลยและแนวคิด (Answer Key)

เอกสารส่วนนี้สำหรับผู้สอน หรือใช้ประเมินตนเองหลังทำข้อสอบ

Part 1: Multiple Choice Questions

1. b. 2NF ระบุว่าต้องไม่มี Partial Dependency
2. d. RA เป็น Procedural Language
3. b. หาพนักงานที่เงินเดือนมากกว่าหรือเท่ากับทุกคน
4. c. Outer Join ทำให้เกิด Null ในตารางผลลัพธ์
5. b. Predicate Pushdown ดันเงื่อนไขให้กรองข้อมูลให้เล็กลงก่อน Join
6. b. Horizontal Partitioning เพราะเป็นการหั่นตารางตามแนวนอน (แถว) ให้เล็กลง
7. a. ทำให้ SELECT เร็วขึ้น แต่กระทบความเร็ว DML เพราะต้องตามไปอัปเดต Index ด้วย
8. d. Cartesian Product สร้าง $N \times M$ records
9. b. EXISTS เช็คระยะว่ามีชุดข้อมูลถูกคืนกลับมาหรือไม่
10. b. เพิ่ม Data Transfer และลดโอกาส Projection Push-down
11. a. ข้อมูลสูญหาย พาลทำให้แม่หาย (Deletion Anomaly)
12. b. WHERE กรองก่อนจัดกลุ่ม HAVING กรองหลังจัดกลุ่ม
13. c. TRUE AND UNKNOWN = UNKNOWN
14. b. NULL = ไม่รู้ค่า เทียบด้วย = ไม่ได้ ต้องใช้ IS NULL
15. c. แก้ปัญหาตัวกำหนดค่าที่ไม่ได้เป็น Candidate Key
16. b. NOT IN ที่เจอ NULL จะคืนค่า UNKNOWN ทำให้ตรรกะที่นั่นทั้ง ไม่ส่งผลลัพธ์ใดๆ กลับมา
17. b. ช่วยซ่อนคอลัมน์ (Security / Data Abstraction)
18. b. UNION ดัดข้อมูลซ้ำถึง UNION ALL ไม่ได้
19. c. > ALL หมายถึงมากกว่าทุกคนในกลุ่มคำตอบ
20. c. DRC อ้างอิงถึงตัวแปรในระดับ Domain (Column)

Part 2: Short Answer Questions

1. IN เทียบค่ากับเซตของข้อมูลว่ามีค่านั้นหรือไม่ (Value matching) ส่วน EXISTS ตรวจสอบแค่ว่ามีค่านั้นคืนค่าเรคคอร์ดกลับมาหรือไม่ (Boolean/Row existence check)
2. > ALL
3. IN
4. Correlated Subquery ต้องอ้างอิงค่าจากตัวแปรใน Outer Query ทำให้ต้องถูกดึงไปประมวลผลซ้ำใหม่ในทุกๆ แถวที่ Outer Query อ่านข้อมูลมา
5. NOT EXISTS
6. วิวนั้นต้องไม่มีการใช้ฟังก์ชัน Aggregate (เช่น SUM, AVG), GROUP BY, หรือ DISTINCT (บาง RDBMS ไม่อนุญาต JOIN ที่ซับซ้อน)
7. 1NF (ข้อมูลต้องเป็น Atomic Value)
8. เป็นการที่แอตทริบิวต์ที่ไม่ใช่คีย์หลัก (Non-prime) ไปทำตัวเป็นข้อจำกัดค่าของแอตทริบิวต์อื่นที่ไม่ใช่คีย์หลักเสียเอง ($A \rightarrow B \rightarrow C$)

9. กรณีที่แอตทริบิวต์รองซ้ำกำหนดค่า (Non-prime determinant) ย้อนกลับไปกำหนดค่าขึ้นส่วนใดขึ้นส่วนหนึ่งของ Composite Primary Key ได้
10. 1) ทำให้การเขียนข้อมูล (INSERT/UPDATE/DELETE) ช้าลงเพราะต้องอัปเดต Index B-Tree ตาม 2) สิ้นเปลืองพื้นที่ Storage

Part 3: Long Questions

ข้อที่ 1.1: หาพนักงานที่ทำงานในโปรเจกต์ 'ProductX'

RA:

$$\pi_{Fname, Lname}((\sigma_{Pname='ProductX'}(PROJECT)) \bowtie WORKS_ON \bowtie EMPLOYEE)$$

TRC:

$$\{t.Fname, t.Lname \mid EMPLOYEE(t) \wedge ((\exists w)(\exists p) (WORKS_ON(w) \wedge PROJECT(p) \wedge p.Pname = 'ProductX' \wedge p.Pnumber = w.Pno \wedge w.Essn = t.Ssn))\}$$

DRC:

$$\{fn, ln \mid (\exists s)(EMPLOYEE(s, fn, _, ln, _, _, _, _)) \wedge (\exists pn)(PROJECT(pn, 'ProductX', _, _)) \wedge WORKS_ON(s, pn, _))\}$$

SQL Statement:

```
1 SELECT E.Fname, E.Lname
2 FROM EMPLOYEE E, WORKS_ON W, PROJECT P
3 WHERE E.Ssn = W.Essn
4       AND W.Pno = P.Pnumber
5       AND P.Pname = 'ProductX';
```

ข้อที่ 1.2: หาพนักงานที่ทำงานใน ทุกๆ โปรเจกต์ ควบคุมโดยแผนก 5 (Division)

RA:

$$E_{all} = \pi_{Ssn}(EMPLOYEE) \times \pi_{Pnumber}(\sigma_{Dnum=5}(PROJECT))$$

$$E_{work} = \pi_{Essn, Pno}(WORKS_ON)$$

$$E_{ans} = \pi_{Ssn}(EMPLOYEE) - \pi_{Ssn}(E_{all} - E_{work})$$

$$\pi_{Fname, Lname}(EMPLOYEE \bowtie E_{ans})$$

หรือใช้ Division:

$$\pi_{Fname, Lname}(EMPLOYEE \bowtie (\pi_{Essn, Pno}(WORKS_ON)$$

$$\div \pi_{Pnumber}(\sigma_{Dnum=5}(PROJECT)))$$

TRC:

$$\{e.Fname, e.Lname \mid e \in EMPLOYEE \wedge \forall p \in PROJECT (p.Dnum = 5 \rightarrow \exists w \in WORKS_ON(w.Pno = p.Pnumber \wedge w.Essn = e.Ssn))\}$$

DRC:

$$\{FN, LN \mid \exists Ssn, \dots (EMPLOYEE(Ssn, FN, \dots)) \wedge \forall Pno, \dots (\neg PROJECT(Pno, \dots, 5) \vee \exists Hrs (WORKS_ON(Ssn, Pno, Hrs)))\}$$

SQL Statement:

```
1 SELECT Fname, Lname
2 FROM EMPLOYEE E
```

```

3 WHERE NOT EXISTS (
4     SELECT Pnumber FROM PROJECT P
5     WHERE Dnum = 5 AND NOT EXISTS (
6         SELECT 1 FROM WORKS_ON W
7         WHERE W.Pno = P.Pnumber AND W.Essn = E.Ssn
8     )
9 );

```

ข้อที่ 1.3: หาพนักงานที่ไม่มีผู้อยู่ในอุปการะ (No Dependents)

RA:

$$E_{\text{has_dep}} = \pi_{\text{Essn}}(\text{DEPENDENT})$$

$$E_{\text{no_dep}} = \pi_{\text{Ssn}}(\text{EMPLOYEE}) - E_{\text{has_dep}}$$

$$\pi_{\text{Fname, Lname}}(\text{EMPLOYEE} \bowtie E_{\text{no_dep}})$$

TRC:

$$\{e.\text{Fname}, e.\text{Lname} \mid e \in \text{EMPLOYEE} \wedge \neg(\exists d \in \text{DEPENDENT}(d.\text{Essn} = e.\text{Ssn}))\}$$

DRC:

$$\{FN, LN \mid \exists Ssn, \dots (\text{EMPLOYEE}(Ssn, FN, \dots) \wedge \neg(\exists Dname, \dots (\text{DEPENDENT}(Ssn, Dname, \dots))))\}$$

SQL Statement:

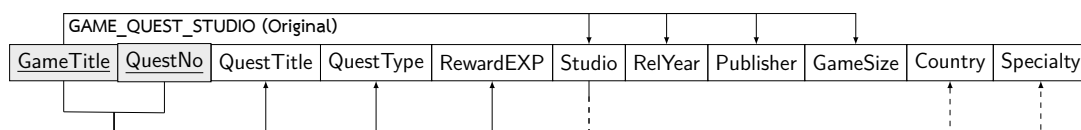
```

1 SELECT Fname, Lname
2 FROM EMPLOYEE E
3 WHERE NOT EXISTS (
4     SELECT 1 FROM DEPENDENT D
5     WHERE D.Essn = E.Ssn
6 );

```

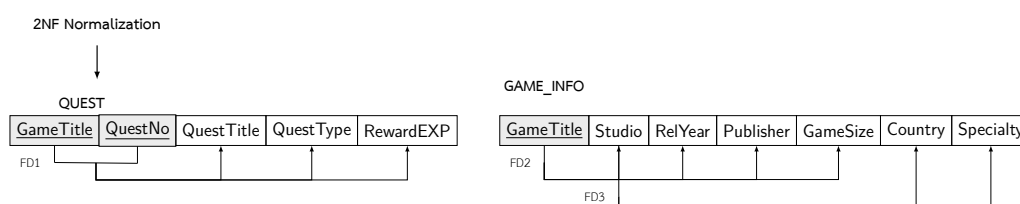
ข้อที่ 2: Normalization

แผนภาพแสดงความสัมพันธ์ (Functional Dependency Diagram) ก่อนเริ่มทำ:



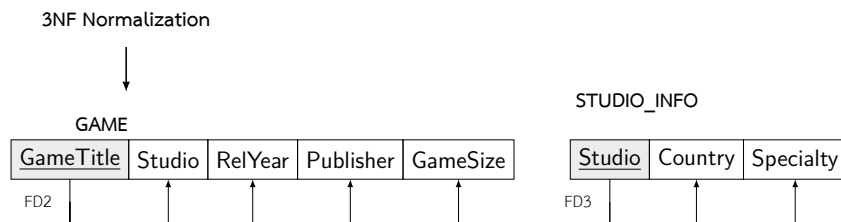
2.1 สถานะ Normal Form (3 คะแนน): ตาราง GAME_QUESTION_STUDIO จัดว่าอยู่ใน 1NF เท่านั้น เนื่องจากมี Partial Dependency (ขัดกับกฎ 2NF) อธิบาย: $\text{GameTitle} \rightarrow \text{Studio}, \text{RelYear}, \text{Publisher}, \text{GameSize}$ (FD2) เป็นการพึ่งพาสายหลักที่ไม่ได้ขึ้นกับ Primary Key ทั้งก่อน (ไม่ได้ขึ้นกับ QuestNo)

2.2 แบ่งตารางเป็น 2NF (3 คะแนน): ต้องแยกแอตทริบิวต์ที่เกิด Partial Dependency ออกจากคีย์หลัก:



- QUEST (GameTitle (FK), QuestNo, QuestTitle, QuestType, RewardEXP)
- GAME_INFO (GameTitle, Studio, RelYear, Publisher, GameSize, Country, Specialty)

2.3 แบ่งตารางเป็น 3NF (4 คะแนน): ตาราง GAME_INFO ใน 2NF มีปัญหา Transitive Dependency เพราะ Studio → Country, Specialty (FD3) ข้อมูลประเทศกับความถนัดขึ้นอยู่กับแค่ตัวสตูดิโอ ไม่ได้ขึ้นอยู่กับชื่อเกม จึงต้องแยกตารางสตูดิโอออกมา:



ผลลัพธ์สุดท้าย (Schema ของ 3NF):

- QUEST (GameTitle (FK), QuestNo, QuestTitle, QuestType, RewardEXP)
- GAME (GameTitle, Studio (FK), RelYear, Publisher, GameSize)
- STUDIO_INFO (Studio, Country, Specialty)

ตัวอย่างข้อมูลในตารางใหม่ที่ถูกแยก (STUDIO_INFO):

Studio (PK)	Country	Specialty
Bethesda	USA	RPG
CDPR	Poland	ARPG
Larian	Belgium	CRPG